

CC_OpenCl.dll

Enterprise-Treiberdokumentation

Architektur · Build · API · Betrieb · Erweiterung

Dokumentstatus: Enterprise-Dokumentation für AMD/MSVC OpenCL Build

Zielartefakt: CC_OpenCl.dll

Dokumentstatus: Enterprise-Dokumentation für den aktuellen AMD/MSVC-Build

Zielartefakt: CC_OpenCL.dll

Buildprofil: Windows x64, MSVC, C++17, OpenCL 3.0 Headers, AMD OpenCL Runtime

Quellkern: src/CC_OpenCL.c

Öffentliche C-Schnittstelle: include/SymBio_Interface.h

Zweck: Wiederverwendbarer GPU-Treiber für zukünftige Projekte mit OpenCL-basierten Tensor-, Simulations-, Agenten-, Mycelium-, SubQG-, Quantum-inspired- und Enterprise-Algorithmusfunktionen.

1. Executive Summary

CC_OpenCL.dll ist eine native GPU-Runtime, die höhere Projektkomponenten von OpenCL-Details entkoppelt. Der Treiber übernimmt Geräteinitialisierung, Speicherverwaltung, Kernelkompilierung, Kernelstart, Messpuffer, Zustandsverwaltung und den Export einer stabilen C-ABI.

Der Treiber ist bewusst nicht nur ein OpenCL-Hilfswrapper. Er kombiniert mehrere Algorithmusdomänen:

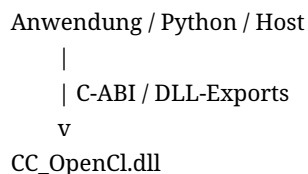
- numerische Tensor- und Deep-Learning-Operationen,
- Bild-/Patch-Transformationen,
- Agenten- und Schwarmfelder,
- Mycelium-/Pheromon-/Koloniedynamik,
- SubQG-Multifield-Simulation,
- neuronale Plastizität und bioinspirierte Updates,
- Quantum-inspired Demonstratoren und SQSE,
- kognitive Signaloperatoren wie Qualia-, Kontext- und Narrativ-Resonanz,
- Enterprise Algorithm Pack mit Resonanzfeld, Energy-Gated Scheduling und Morphogenese.

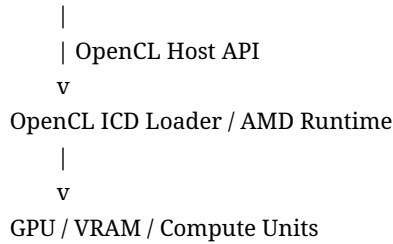
Die wichtigste architektonische Idee ist: Daten bleiben nach Möglichkeit im VRAM. Die Hostseite ruft Exportfunktionen auf, übergibt Bufferhandles, und der Treiber verarbeitet diese Buffers direkt über OpenCL-Kernel. Dadurch wird der teuerste Fehler vieler GPU-Projekte vermieden: ständiges Kopieren zwischen Host und Device.

2. Systemrolle und Designphilosophie

2.1 Rolle im Gesamtsystem

Der Treiber bildet die native Ausführungsschicht zwischen einer High-Level-Anwendung und der GPU.





Die Hostanwendung soll keine Kernelstrings, Workgroupgrößen, clSetKernelArg-Folgen oder OpenCL-Programmobjekte verwalten müssen. Sie arbeitet mit:

- initialize_gpu(...),
- allocate_gpu_memory(...),
- write_host_to_gpu_blocking(...),
- algorithmischen Execute-Funktionen,
- read_gpu_to_host_blocking(...),
- shutdown_gpu(...).

2.2 Warum eine DLL statt reinem Python?

Python ist hervorragend für Steuerung, Modellierung und Experimentieren. Für große Felder, Tensoren und Agentensysteme entstehen aber drei typische Engpässe:

1. **Python-Objektkosten:** Millionen Agenten als Python-Objekte erzeugen hohe Speicher- und Dispatchkosten.
2. **GIL und CPU-Schleifen:** lokale Nachbarschaftsdynamik skaliert schlecht in Python-Schleifen.
3. **Host-Device-Kopien:** Daten, die pro Tick zwischen CPU und GPU wandern, zerstören Performance.

Die DLL verschiebt diese Engpässe:

Engpass	Verschiebung im Treiber
Python-Schleifen	OpenCL-Kernel über globale Work-Items
Python-Objekte	flache Buffer und strukturierte Felder
GIL	native DLL + GPU-Ausführung
Host-Device-Kopien	persistente VRAM-Buffer
uneinheitliche Algorithmen	einheitliche C-ABI

2.3 Enterprise-Prinzipien

Der Treiber folgt diesen Betriebsprinzipien:

- **Explizite Initialisierung:** jedes GPU-Gerät wird über initialize_gpu(gpu_index) vorbereitet.
- **Explizite Bufferbesitzverhältnisse:** GPU-Speicher wird über Handles verwaltet.
- **ABI-Stabilität:** Exportfunktionen verwenden C-kompatible Typen.
- **Fehlerabfrage:** Diagnose über cc_get_last_error().
- **Trennung von Build und Runtime:** OpenCL.lib ist Link-Artefakt, OpenCL.dll kommt zur Laufzeit aus der OpenCL-Installation.
- **Algorithmische Erweiterbarkeit:** neue Algorithmusgruppen können als Kernelpack ergänzt werden.
- **Messbarkeit:** neue Algorithmen müssen benchmarkbar sein.

3. Build- und Deployment-Modell

3.1 Empfohlenes Buildprofil

Für Windows/AMD ist der direkte MSVC-Build das robusteste Profil:

```
cd /d D:\CC_OpenCL_Driver_Enterprise
scripts\build_windows_msvc_amd_direct.bat
```

Voraussetzungen:

- Visual Studio Build Tools 2022,
- x64 Native Tools Command Prompt for VS 2022,
- cl.exe,
- lib.exe,
- AMD-Grafiktreiber mit OpenCL-Runtime,
- vollständiger CL-Headerordner,
- CL\OpenCL.def oder vorhandene build\OpenCL.lib.

3.2 Warum MSVC/C++17?

Die Datei heißt zwar CC_OpenCL.c, enthält aber OpenCL-Kernel als C++ Raw-String-Literale:

```
R"CLC(
__kernel void ...
)CLC"
```

Diese Syntax ist C++, nicht C. Deshalb wird die Datei durch das Buildsript mit /TP /std:c++17 kompiliert. /TP zwingt MSVC, die .c-Datei als C++ zu behandeln. Die DLL-Exports bleiben trotzdem C-kompatibel, wenn sie korrekt über extern "C" / Exportmakros geführt werden.

3.3 OpenCL-Headers und AMD Runtime

Für den Build werden Header wie CL/cl.h, CL/cl_platform.h, CL/cl_ext.h benötigt. Für das Linken wird eine x64-Importbibliothek OpenCL.lib verwendet. Bei AMD kann sie aus CL\OpenCL.def erzeugt werden:

```
lib /def:CL\OpenCL.def /machine:x64 /out:build\OpenCL.lib
```

Zur Laufzeit muss die OpenCL-ICD vorhanden sein. Unter Windows ist ein einfacher Check:

```
dir C:\Windows\System32\OpenCL.dll
```

3.4 Build-Artefakte

Nach erfolgreichem Build:

```
build\CC_OpenCL.dll  Hauptartefakt für Runtime
build\CC_OpenCL.lib  Importbibliothek für C/C++-Linker
build\CC_OpenCL.exp  Exportinformationen
build\OpenCL.lib     OpenCL-Importbibliothek
```

3.5 Deployment-Regel

Für High-Level-Projekte genügt typischerweise:

```
CC_OpenCL.dll
```

Für native C/C++-Projekte, die direkt gegen die DLL linken:

```
CC_OpenCL.dll
CC_OpenCL.lib
include\SymBio_Interface.h
```

Die OpenCL-Runtime wird nicht mitgeliefert, sondern über den GPU-Treiber bereitgestellt.

4. Laufzeitarchitektur

4.1 Initialisierungspfad

Typischer Ablauf:

```
initialize_gpu(gpu_index)
-> Plattform/Gerät auswählen
-> Kontext erzeugen
-> Command Queue erzeugen
-> Kernels/Programme vorbereiten
-> Slotstatus speichern
```

Danach können Buffer angelegt und Algorithmen ausgeführt werden.

4.2 Speicherpfad

```
allocate_gpu_memory
-> clCreateBuffer
write_host_to_gpu_blocking
-> clEnqueueWriteBuffer
execute_...
-> clSetKernelArg + clEnqueueNDRangeKernel
read_gpu_to_host_blocking
-> clEnqueueReadBuffer
free_gpu_memory
-> clReleaseMemObject
```

4.3 Slotmodell

Der Treiber arbeitet mit `gpu_index`. Dadurch kann er mehrere GPUs logisch unterscheiden. Jede GPU wird als Slot verwaltet:

```
GpuSlot
context
device_id
command_queue
program/kernel handles
domain-specific buffers
metrics/status
```

Für Enterprise-Betrieb heißt das: eine Anwendung sollte für jedes verwendete Gerät explizit initialisieren und am Ende explizit freigeben.

4.4 Fehlerbehandlung

Viele Exportfunktionen liefern int zurück:

- 1 oder positiver Wert: Erfolg,
- 0 oder negativer Wert: Fehler, abhängig von Funktion.

Die Detailmeldung wird über `cc_get_last_error()` abgefragt.

Empfohlenes Hostmuster:

```
if (!execute_resonant_field_step_gpu(...)) {  
    fprintf(stderr, "CC_OpenCl error: %s\n", cc_get_last_error());  
}
```

4.5 Synchronisation

Der Treiber enthält sowohl blockierende Transferfunktionen als auch Kernelstarts. Für reproduzierbare Tests sollte nach einer Sequenz `finish_gpu(gpu_index)` verwendet werden, falls eine Funktion nicht selbst blockiert.

5. Hauptmodule des Treibers

5.1 Runtime, Speicher und Diagnose

Dieses Modul stellt den Grundbetrieb bereit:

- GPU initialisieren und schließen,
- Speicher auf der GPU anlegen und freigeben,
- Daten schreiben und lesen,
- Fehler und Versionsinformationen abfragen,
- Kernelmessungen lesen.

Wichtige Funktionen:

```
initialize_gpu  
allocate_gpu_memory  
free_gpu_memory  
write_host_to_gpu_blocking  
read_gpu_to_host_blocking  
finish_gpu  
shutdown_gpu  
cc_get_last_error  
cc_get_version  
get_last_kernel_metrics
```

5.2 Tensor- und Deep-Learning-Kernel

Der Treiber enthält Grundoperationen für neuronale Modelle:

- Matrixmultiplikation,
- batched Matmul,
- Softmax,

- GELU,
- Elementwise Add/Mul,
- LayerNorm und Backward,
- Adam Update,
- Conv2D Forward/Backward,
- Transpose,
- Clone,
- Embeddings,
- CTC-Loss CPU-seitig.

Diese Funktionen erlauben, Modelle oder Teilmodelle direkt im Treiber auszuführen, ohne auf externe Frameworks angewiesen zu sein.

Typische Einsatzfälle:

- kleine spezialisierte Inferenzpfade,
- eigene Lernschleifen,
- GPU-nahe Prototypen,
- Tests neuer Datenlayouts.

5.3 Vision-, Patch- und Encoderpfade

Enthalten sind:

- Patch-Permute-Reshape,
- Rückwärtsoperationen für Patch-Layout,
- EON-Encoder-Ketten,
- dynamische Tokenzuweisung.

Diese Funktionen sind nützlich, wenn Bild-/Featurefelder in Tokenstrukturen umgewandelt werden sollen, ohne über CPU-Speicher zu gehen.

5.4 Mycelium-, Pheromon- und Kolonie-System

Dieses Modul modelliert lokale Feld- und Agentendynamik:

- Pheromonverstärkung,
- Diffusion/Zerfall,
- Nährstoffupdate,
- Mycelium-Update,
- Kolonieupdate,
- Reproduktion,
- Mood- und Nutrient-State,
- Nachbarschaftstabellen.

Die Datenstruktur ist bewusst feldbasiert statt objektbasiert:

```
pheromone[T][K][C]
nutrient[T]
mood[T][C]
neighbors[T][K]
```

colonies[T]
potential[T]

Dadurch kann ein Work-Item pro Zelle oder Knoten arbeiten.

5.5 SubQG- und Multifield-Simulation

SubQG verwaltet mehrere gekoppelte Felder:

- Energie,
- Druck,
- Gravitation,
- Magnetismus,
- Temperatur,
- Potential,
- Drift X/Y.

Das Hostinterface nutzt `subqg_set_multifield_state` und `subqg_get_multifield_state`, um komplette Feldzustände zu laden oder zu exportieren.

Dieses Modul eignet sich als generische Feldsimulation, Debugbasis und Kopplungsschicht für Mycelium und Agenten.

5.6 Agenten- und bioinspirierte Lernsysteme

Der Treiber enthält mehrere Mechanismen für Agentensysteme:

- genetische Agentenupdates,
- Agenteninjektion in SubQG-Felder,
- Hebbian Learning,
- Izhikevich-Neuronen,
- STDP-Traces,
- Agentenzyklen,
- VRAM-Organismus-Zyklen.

Die Leitidee ist, Agenten nicht als Hostobjekte zu verarbeiten, sondern als Felder oder flache Strukturen im GPU-Speicher.

5.7 Quantum-inspired und SQSE

Das Quantum-Modul bietet experimentelle, quantum-inspired Operationen:

- Shor-Demonstrator,
- Grover-Demonstrator,
- VQE,
- QAOA,
- HHL,
- QML Classifier,
- QEC Cycle,
- Gate-Sequence Upload/Apply,
- QASM Import/Export,
- OTOC/Echo-Analyse,
- SQSE Encrypt/Decrypt.

Diese Funktionen sind als Forschungsschicht zu betrachten. Sie ersetzen keinen echten Quantencomputer, stellen aber GPU-basierte Simulationen, Heuristiken und Signaloperatoren bereit.

5.8 Cognitive / Narrative Signal Operators

Dieses Modul verarbeitet Signale als Resonanz- und Abstraktionsfelder:

- Qualia-Resonanz,
- Intuition/Precognition,
- Kontext-Resonanz,
- Dream-State-Generierung,
- Transformationsplanung,
- Systemnarrativ,
- symbolische Konzeptabstraktion.

Enterprise-Hinweis: Diese Begriffe sind im Treiber technische Signaloperationen, keine Garantie für semantisches Bewusstsein. Sie sollten als Feature-Engineering- und Signaltransformationskernel dokumentiert und getestet werden.

5.9 Enterprise Algorithm Pack

Der Enterprise Algorithm Pack erweitert den Treiber um drei generische Bausteine:

4. Resonant Field Propagation,
5. Energy-Gated Agent Scheduler,
6. Morphogenetic Rule Kernel.

Diese Bausteine sind bewusst generisch, damit spätere Projekte sie für Mycelium, Agenten, Scheduling, Feldsimulationen oder adaptive Systeme nutzen können.

6. Enterprise Algorithm Pack im Detail

6.1 Resonant Field Propagation

Zweck

Ein gekoppelte-Oszillator-Feld über einem fixed-K Nachbarschaftsgraphen.

Datenmodell

state[N] aktueller Feldzustand
velocity[N] Feldgeschwindigkeit / Trägheit
drive[N] externer Antrieb
energy[N] Ausgabesignal für Aktivität/Spannung
neighbors[N*K] Nachbarindizes
weights[N*K] Kopplungsgewichte

Updategleichung

Für jeden Knoten i:

$\text{gradient} = \text{gewichtete Differenz zu Nachbarn}$
 $\text{velocity} = \text{velocity} * \text{inertia} + \text{coupling} * \text{gradient} + \text{drive} - \text{damping} * \text{state}$
 $\text{state} = \text{state} + \text{dt} * \text{velocity}$
 $\text{energy} = \text{abs}(\text{velocity}) + \text{abs}(\text{gradient})$

Einsatz

- Aktivierungsfeld für Agentensysteme,
- Prioritätsfeld für Scheduler,
- Pheromon- oder Potentialverstärker,
- Stabilitätsanalyse,
- kontinuierliche Simulation lokaler Wechselwirkungen.

Exportfunktion

```

int execute_resonant_field_step_gpu(
    int gpu_index,
    void* state_buf,
    void* velocity_buf,
    void* drive_buf,
    void* energy_buf,
    void* neighbors_buf,
    void* weights_buf,
    int N,
    int K,
    float dt,
    float damping,
    float coupling,
    float inertia,
    float clamp_abs
);

```

Parameterempfehlung

Parameter	Startwert	Bedeutung
dt	0.01 bis 0.1	Schrittweite
damping	0.01 bis 0.2	Rückstell-/Dämpfungskraft
coupling	0.1 bis 1.0	Kopplung an Nachbarn
inertia	0.8 bis 0.99	Gedächtnis der Geschwindigkeit
clamp_abs	1.0 bis 100.0	Schutz vor Explosion

6.2 Energy-Gated Agent Scheduler

Zweck

Nicht alle Agenten werden voll aktualisiert. Aktivität wird GPU-seitig aus einem Energiefeld abgeleitet.

Datenmodell

```

energy[N]
nutrient[N]
active_flags[N]
active_indices[N]
active_count[1]

```

Ablauf

```
active_count = 0
cc_mark_active_agents(...)
cc_gated_agent_decay(...)
```

active_indices kann von nachfolgenden Agentenkernen genutzt werden, um nur aktive Agenten zu verarbeiten.

Exportfunktion

```
int execute_energy_gated_scheduler_gpu(
    int gpu_index,
    void* energy_buf,
    void* nutrient_buf,
    void* active_flags_buf,
    void* active_indices_buf,
    void* active_count_buf,
    int N,
    float threshold,
    float sleep_decay,
    float nutrient_recovery
);
```

Vorteil

Bei geringer Aktivitätsrate kann der teure Teil des Agentenupdates von $O(N)$ auf $O(M)$ verschoben werden, wobei M die Anzahl aktiver Agenten ist. Der Scan bleibt $O(N)$, ist aber billig.

Risiko

Wenn fast alle Agenten aktiv sind, verursacht das Gating zusätzliche Arbeit. Dann sollte ein Full-Step genutzt werden.

6.3 Morphogenetic Rule Kernel

Zweck

Regelbasierte Zell-/Strukturentwicklung direkt auf der GPU.

Datenmodell

```
cell_type[N]
nutrient[N]
energy[N]
potential[N]

rule_in_type[R]
rule_min_nutrient[R]
rule_min_energy[R]
rule_out_type[R]
rule_delta_potential[R]
```

Semantik

Jede Zelle prüft Regeln in Prioritätsreihenfolge. Die erste passende Regel verändert Zelltyp, Potential und Nährstoff.

Exportfunktion

```
int execute_morphogenetic_rule_step_gpu(
    int gpu_index,
```

```

void* cell_type_buf,
void* nutrient_buf,
void* energy_buf,
void* potential_buf,
void* rule_in_type_buf,
void* rule_min_nutrient_buf,
void* rule_min_energy_buf,
void* rule_out_type_buf,
void* rule_delta_potential_buf,
int N,
int R,
float nutrient_cost
);

```

Einsatz

- Zelluläre Automaten mit kontinuierlichen Feldern,
- Morphogenese von Koloniestrukturen,
- adaptive Zonierung,
- GPU-seitige Regelmaschinen.

7. Empfohlenes Integrationsmuster

7.1 C/C++ Host

```

initialize_gpu(0);

void* state = allocate_gpu_memory(0, N * sizeof(float));
void* velocity = allocate_gpu_memory(0, N * sizeof(float));
void* drive = allocate_gpu_memory(0, N * sizeof(float));
void* energy = allocate_gpu_memory(0, N * sizeof(float));

write_host_to_gpu_blocking(0, state, 0, N * sizeof(float), host_state);

execute_resonant_field_step_gpu(
    0,
    state,
    velocity,
    drive,
    energy,
    neighbors,
    weights,
    N,
    K,
    0.05f,
    0.05f,
    0.8f,
    0.95f,
    10.0f
);

```

```

read_gpu_to_host_blocking(0, energy, 0, N * sizeof(float), host_energy);

free_gpu_memory(0, state);
free_gpu_memory(0, velocity);
free_gpu_memory(0, drive);
free_gpu_memory(0, energy);
shutdown_gpu(0);

```

7.2 Python / ctypes Host

Für Python-Integration sollte die DLL per ctypes.CDLL geladen werden. Wichtig ist, argtypes und restype sauber zu setzen. GPU-Bufferhandles werden als ctypes.c_void_p geführt.

Empfohlenes Pattern:

```

from ctypes import CDLL, c_int, c_void_p, c_size_t, c_float

dll = CDLL("CC_OpenCl.dll")
dll.initialize_gpu.argtypes = [c_int]
dll.initialize_gpu.restype = c_int

dll.allocate_gpu_memory.argtypes = [c_int, c_size_t]
dll.allocate_gpu_memory.restype = c_void_p

dll.execute_resonant_field_step_gpu.argtypes = [
    c_int,
    c_void_p, c_void_p, c_void_p, c_void_p, c_void_p, c_void_p,
    c_int, c_int,
    c_float, c_float, c_float, c_float, c_float,
]
dll.execute_resonant_field_step_gpu.restype = c_int

```

7.3 Best Practice: Buffer persistent halten

Schlecht:

```

pro Tick:
    allocate
    write
    execute
    read
    free

```

Gut:

```

einmal:
    allocate
    initial write

```

```

pro Tick:
    execute
    optional small readback

```

am Ende:
 read final state
 free

8. Qualitätssicherung und Tests

8.1 Buildtest

Muss nach jedem Commit laufen:

scripts\build_windows_msvc_amd_direct.bat

Erfolgskriterien:

build\CC_OpenCl.dll existiert
keine fatal errors
keine neuen Compilerwarnungen ohne Review

8.2 Smoke-Test

Minimal:

7. DLL laden.
8. cc_get_version() aufrufen.
9. initialize_gpu(0) aufrufen.
10. kleinen Buffer allozieren.
11. Hostdaten schreiben.
12. einfachen Kernel ausführen, z. B. Add oder GELU.
13. Ergebnis lesen.
14. Speicher freigeben.
15. shutdown_gpu(0).

8.3 Numerische Tests

Für jeden Kernel sollte ein CPU-Referenztest existieren:

Klasse	Referenz
Elementwise	CPU-Schleife
Matmul	naive CPU-Matmul für kleine Größen
Resonanzfeld	CPU fixed-K Graphstep
Morphogenese	CPU-Regelmaschine
Scheduler	CPU-Aktivitätsliste
Diffusion	kleine deterministische Nachbarschaft

8.4 Stabilitätstests

- $N = 0$, $N = 1$, kleine Randfälle,
- falsche Buffergrößen sollen vermieden oder klar dokumentiert werden,
- NaN-/Inf-Eingaben,
- große N ,
- mehrere Initialisierung/Shutdown-Zyklen,

- mehrfacher Build und DLL-Replacement.

8.5 Performance-Benchmark

Empfohlene Metriken:

- Kernelzeit in ms,
- effektive Speicherbandbreite,
- Host-Device-Transferzeit,
- First-call-Kompilierkosten,
- Wiederholungszeit nach Warmup,
- VRAM-Verbrauch,
- aktive Agentenquote bei Gated Scheduler.

Benchmarkregel:

Warmup nicht mitmessen.

Transfers separat messen.

finish_gpu nach Messabschnitt nutzen.

CPU-Referenz nur für kleine Größen validieren.

9. Betrieb, Monitoring und Fehleranalyse

9.1 Fehlerquellen

Symptom	Wahrscheinliche Ursache	Maßnahme
CL/cl.h fehlt	Headerordner unvollständig	Khronos OpenCL-Headers kopieren
OpenCL.lib fehlt	Importlib nicht erzeugt	lib /def:CL\OpenCL.def ...
DLL lädt nicht	Runtime fehlt oder PATH falsch	OpenCL.dll prüfen
Kernelbuild schlägt fehl	OpenCL-Sourcefehler	Buildlog über Last Error prüfen
Ergebnis ist Null	Buffer nicht geschrieben oder falsche Größe	Host-Transfer prüfen
GPU hängt	zu große Workload oder instabiler Kernel	kleinere N, Timeout beachten
NaN-Ausbreitung	Parameter instabil	Clamp/Damping erhöhen
Gated Scheduler langsamer	Aktivitätsrate zu hoch	Full-Step nutzen

9.2 AMD-spezifische Hinweise

- Für Desktop-GPUs kann Windows TDR lange Kernel abbrechen.
- Große Monolith-Kernel sollten in kleinere Schritte zerlegt werden.
- OpenCL-Header-Version und Runtime-Version sind getrennte Themen: Header können OpenCL 3.0 definieren, Runtime kann je nach Treiber andere Fähigkeiten melden.
- Immer über Runtime-Geräteabfrage validieren, welche Features verfügbar sind.

9.3 Release-Checkliste

Vor einem Release:

- frischer Build aus sauberem Ordner,
- DLL-Größe und Timestamp prüfen,
- dumpbin /exports build\CC_OpenCl.dll prüfen,
- Smoke-Test durchführen,

- API-Header mit DLL-Exports abgleichen,
- README.md und Dokumentation aktualisieren,
- Changelog schreiben,
- ZIP mit Quellcode, Headern, Skripten, Docs und Buildartefakten trennen.

10. Sicherheits- und Robustheitsmodell

10.1 ABI-Grenzen

Die DLL kann nicht wissen, ob ein `void*` wirklich ein gültiges `cl_mem`-Handle ist. Hostcode muss Buffer korrekt erzeugen und Größen einhalten.

10.2 Speichergrößen

Die meisten Funktionen erhalten Längenparameter. Diese müssen mit den wirklich allokierten Buffern übereinstimmen. Falsche Größen können zu OpenCL-Fehlern oder undefiniertem Verhalten führen.

10.3 Threading

Nicht jede Exportfunktion ist automatisch thread-safe. Für Enterprise-Einsatz gilt:

- pro GPU-Slot serialisieren oder Locking einführen,
- keine parallelen `shutdown_gpu`-Aufrufe während laufender Kernels,
- keine Buffer freigeben, während sie in einer Queue benutzt werden.

10.4 Determinismus

Einige agentische oder quantum-inspired Funktionen nutzen bewusst GPU-Parallelität, Atomics oder nichtdeterministische Schedulingeffekte. Für reproduzierbare Tests sollten deterministische Modi, feste Seeds und kleine Größen genutzt werden.

11. Architekturentscheidungen und Begründung

11.1 Warum flache Buffer?

Flache Buffer sind GPU-freundlich, ABI-stabil und sprachübergreifend nutzbar. Sie vermeiden Objektgraphen und erlauben lineares Speichermuster.

11.2 Warum `void*` für GPU-Buffer?

`void*` kapselt `cl_mem`, ohne OpenCL-Typen in jede Hostsprache zu zwingen. Für Python/ctypes, C# P/Invoke oder andere FFI-Schichten ist das einfacher. Das Risiko ist geringere Typsicherheit.

11.3 Warum ein monolithischer Kern plus Algorithm Pack?

Historisch enthält `CC_OpenCL.c` viele Kernel in einer Datei. Für Erweiterungen ist ein Algorithm-Pack-Ansatz besser, weil er neue Mechanismen logisch trennt. Das aktuelle Enterprise Pack ist der erste Schritt in diese Richtung.

11.4 Warum keine Framework-Abhängigkeit?

Der Treiber soll eigenständig bleiben. Externe Frameworks würden Deployment, ABI und Build reproduzierbarkeit erschweren. OpenCL ist die einzige zentrale GPU-Abhängigkeit.

12. Erweiterungsleitfaden

12.1 Neue Funktion hinzufügen

Empfohlener Ablauf:

16. Kernel in eigenen Sourceblock oder Pack einfügen.
17. Host-Wrapper mit klarer Signatur schreiben.
18. Export in Symbio_Interface.h deklarieren.
19. Fehlerbehandlung über cc_set_last_error.
20. Buffergrößen und Nullpointer prüfen.
21. CPU-Referenztest schreiben.
22. Benchmark schreiben.
23. Dokumentation und API-Referenz aktualisieren.

12.2 Namenskonvention

Empfohlen:

```
execute_<domain>_<operation>_gpu  
step_<domain>_<operation>  
read_<domain>_<state>  
set_<domain>_<params>
```

Kernel intern:

```
cc_<domain>_<operation>
```

12.3 Parameterdesign

Besser:

N, K, R, stride, count explizit

Schlechter:

implizite globale Größen ohne Hostvalidierung

12.4 Fehlerbehandlung

Jede neue Exportfunktion sollte:

- gpu_index validieren,
- Bufferpointer prüfen,
- Größen prüfen,
- ensure_*_kernels_ready aufrufen,
- jeden OpenCL-Fehler abfangen,

- im Fehlerfall cc_get_last_error befüllen.

13. Einsatzempfehlungen nach Projekttyp

13.1 Simulationen

Nutze:

- SubQG-Multifield,
- Resonant Field,
- Morphogenetic Rules,
- Diffusion,
- LBM/N-Body/Ising bei Bedarf.

Strategie:

Zustand persistent im VRAM
pro Tick mehrere Kernel
nur Kontrollmetriken zurücklesen

13.2 Agentensysteme

Nutze:

- Energy-Gated Scheduler,
- Mycelium/Pheromone,
- Genetic Agents,
- Hebbian/STDP,
- Resonant Field als Aktivitätsfeld.

Strategie:

Energie -> aktive Liste -> Agentenupdate -> Feldfeedback

13.3 KI-/Deep-Learning-Experimente

Nutze:

- Matmul,
- LayerNorm,
- Softmax/GELU,
- Adam,
- Embedding,
- Patch-Operationen.

Strategie:

kleine spezialisierte Modelle direkt im Treiber
große Standardmodelle nur, wenn kein Framework gebraucht wird

13.4 Forschungssysteme

Nutze:

- Quantum-inspired Module,
- Qualia/Kontext/Narrativ-Operatoren,
- SQSE,
- SubQG-Feedback.

Strategie:

klar zwischen technischer Signaloperation und Interpretation trennen
messbare Hypothesen formulieren

14. Bekannte Grenzen

- Der Treiber ist groß und enthält mehrere experimentelle Domänen.
- Viele APIs arbeiten mit rohen Buffern; Hostfehler sind möglich.
- OpenCL-Fehlerdiagnose hängt vom Runtime-Treiber ab.
- Performance ist geräteabhängig.
- Einige Konzepte sind Forschungsoperatoren und müssen projektspezifisch validiert werden.
- C++17-Build ist erforderlich, obwohl die Datei .c heißt.
- CMake kann in MSYS/Unix-Varianten ungeeignete Generatoren anzeigen; der direkte MSVC-Build ist daher bevorzugt.

15. Empfohlene Roadmap

Phase 1 – Stabilisierung

- alle Warnungen eliminieren,
- Exportliste mit Header abgleichen,
- Smoke-Test automatisieren,
- Release-ZIP standardisieren.

Phase 2 – Testframework

- CPU-Referenzen,
- deterministic mode tests,
- Buffergrößentests,
- Performance-Warmup.

Phase 3 – Pack-Modularisierung

- Enterprise Pack separat verwalten,
- Mycelium Pack separieren,
- Quantum Pack separieren,
- optional Binary-Kernel-Cache.

Phase 4 – Host-SDK

- C-Header finalisieren,
- Python-ctypes Wrapper,
- C# P/Invoke Wrapper,
- Beispielprojekte.

Phase 5 – Production Hardening

- Threadingmodell definieren,
- Error Codes standardisieren,
- Memory-Leak-Tests,
- Device capability checks,
- CI-Build für Windows x64.

16. Kurzreferenz: empfohlenes Hostmuster

1. initialize_gpu
2. allocate_gpu_memory
3. write_host_to_gpu_blocking
4. execute_*_gpu / step_*
5. optional finish_gpu
6. optional read_gpu_to_host_blocking
7. free_gpu_memory
8. shutdown_gpu

17. Entscheidender Nutzen

CC_OpenCl.dll ist ein wiederverwendbarer GPU-Treiber, der nicht nur einzelne Operationen bereitstellt, sondern ein experimentelles Ausführungsmodell ermöglicht:

- Daten als Felder statt Objekte,
- Agenten als kompakte Zustandsarrays,
- Verhalten als GPU-Kernel,
- Scheduling über Energie statt CPU-Listen,
- Morphogenese über Regeltabellen,
- Simulation und KI in einer gemeinsamen VRAM-Laufzeit.

Damit entsteht eine Grundlage für zukünftige Projekte, in denen Python oder andere High-Level-Sprachen nur noch steuern, während die eigentliche Dynamik persistent und parallel auf der GPU läuft.

Anhang A – Exportierte API-Gruppen

Die vollständige maschinennahe API-Referenz liegt zusätzlich als docs/API_REFERENCE.md im Projekt. Die folgende Übersicht gruppiert die Exportfunktionen nach Funktionsdomäne.

Runtime / Speicher / Diagnose

Funktion	Rückgabe	Zeile
set_kernel_blocking	void	1007
set_quantum_enabled	void	2256
cc_get_last_error	const char*	2668
get_last_quantum_echo_profile	int	6321
set_quantum_enabled	void	7264
initialize_gpu	int	7275
free_gpu_memory	void	7859
write_host_to_gpu_blocking	int	7872
read_gpu_to_host_blocking	int	7907
shutdown_gpu	void	13211
finish_gpu	int	16035
cc_get_last_error	const char*	16039
cc_get_version	const char*	16043
get_last_kernel_metrics	int	17742

Tensor / Deep Learning

Funktion	Rückgabe	Zeile
subqg_set_multifield_state	int	8231
subqg_get_multifield_state	int	8266
subqg_simulation_step	int	8366
subqg_simulation_step_batched	int	8556
execute_matmul_on_gpu	int	16437
execute_softmax_on_gpu	int	16459
execute_gelu_on_gpu	int	16466
execute_add_on_gpu	int	16473
execute_add_bias_on_gpu	int	16480
execute_mul_on_gpu	int	16487
execute_conv2d_forward_on_gpu	int	16511
execute_conv2d_backward_on_gpu	int	16553
compute_ctc_loss_cpu	int	16661
execute_layernorm_on_gpu	int	16866
execute_clone_on_gpu	int	16874
execute_transpose_on_gpu	int	16881
execute_gelu_backward_on_gpu	int	16888
execute_matmul_backward_on_gpu	int	16895
execute_layernorm_backward_on_gpu	int	16922
execute_adam_update_on_gpu	int	16930
execute_softmax_backward_on_gpu	int	16955
execute_mul_backward_on_gpu	int	16962
execute_transpose_backward_on_gpu	int	16970
execute_embedding_lookup_gpu	int	16977

execute_embedding_backward_gpu	int	16985
execute_reduce_sum_gpu	int	17024
execute_broadcast_add_gpu	int	17031
execute_transpose_batched_gpu	int	17038
execute_transpose_12_batched_gpu	int	17045
execute_matmul_batched_on_gpu	int	17052

Vision / Patches / Encoder

Funktion	Rückgabe	Zeile
execute_patch_permute_reshape_on_gpu	int	16599
execute_patch_permute_reshape_backward_on_gpu	int	16617
execute_eon_encoder_chain_on_gpu	int	16635
execute_eon_encoder_backward_chain_on_gpu	int	16648
execute_dynamic_token_assignment_gpu	int	17127

Mycelium / Agenten / Kolonie

Funktion	Rückgabe	Zeile
subqg_inject_agents	int	8804
update_genetic_agents	int	8869
subqg_init_mycel	int	9059
subqg_realloc_pheromone_channels	int	9280
subqg_set_nutrient_recovery	int	9358
set_pheromone_gains	int	9372
set_mood_state	int	9443
set_nutrient_state	int	9465
step_pheromone_reinforce	int	9486
step_pheromone_diffuse_decay	int	9565
step_mycel_update	int	9613
step_colony_update	int	9743
step_reproduction	int	9748
read_pheromone_slice	int	9871
read_full_pheromone_buffer	int	9957
read_nutrient	int	10374
save_mycel_state	int	10477
load_mycel_state	int	10530
step_hebbian_social_learning	int	10618
mycel_agent_cycle	int	10673
cycle_vram_organism	int	10896
execute_qec_cycle_gpu	int	12494
execute_hebbian_update_on_gpu	int	17111

SubQG / Multi-Field Simulation

Funktion	Rückgabe	Zeile
subqg_initialize_state_batched	int	1997
subqg_initialize_state	int	7969
subqg_initialize_state_batched	int	7973
subqg_debug_read_field	int	8211
subqg_debug_read_channel	int	8312
subqg_set_active_T	int	9268
subqg_set_repro_params	int	9345
step_subqg_feedback	int	9808
step_potential_for_hpio	int	9839
subqg_get_dims	int	9948
read_potential	int	10388
subqg_set_deterministic_mode	void	10596
subqg_release_state	void	10608
subqg_set_params	void	10901
execute_resonant_field_step_gpu	int	16189

Quanten / SQSE / Quantum-Inspired

Funktion	Rückgabe	Zeile
execute_shor_gpu	int	10950
execute_grover_gpu	int	11057
execute_vqe_gpu	int	11128
execute_vqe_gradients_parallel_gpu	int	11198
execute_qaoa_gpu	int	12324
execute_hhl_gpu	int	12370
execute_qml_classifier_gpu	int	12457
quantum_upload_gate_sequence	int	12617
quantum_apply_gate_sequence	int	12664
quantum_export_to_qasm	int	12773
quantum_import_from_qasm	int	12824
execute_quantum_echoes_otoc_gpu	int	13017
sqse_load_kernels	int	17488
execute_sqse_encrypt_float	int	17505
execute_sqse_decrypt_float	int	17618

Cognitive / Narrative / Qualia

Funktion	Rückgabe	Zeile
compute_qualia_resonance_gpu	int	11367
compute_intuition_precognition_gpu	int	11529
compute_context_resonance_gpu	int	11674
generate_dream_state_gpu	int	11819
plan_transformation_gpu	int	11973
generate_system_narrative_gpu	int	12081
abstract_to_symbolic_concepts_gpu	int	12222

Enterprise Algorithm Pack

Funktion	Rückgabe	Zeile
execute_energy_gated_scheduler_gpu	int	16266
execute_morphogenetic_rule_step_gpu	int	16358

Noise / Metrics / Kernel-Simulation

Funktion	Rückgabe	Zeile
set_noise_level	void	17722
get_noise_level	float	17727
register_kernel_measurement_buffers	void	17732
reset_kernel_measurement_buffers	void	17737

Sonstige

Funktion	Rückgabe	Zeile
set_diffusion_params	int	9397
set_neighbors_sparse	int	9421
render_frame_to_buffer	int	9977
read_colonies	int	10433
launch_shadow_self_reenqueue	int	10918
execute_threshold_spike_on_gpu	int	17120
execute_pairwise_similarity_gpu	int	17135
execute_proto_segmented_sum_gpu	int	17143
execute_proto_update_step_gpu	int	17152
execute_shape_loss_with_reward_penalty_gpu	int	17161
execute_fused_diffusion_on_gpu	int	17195
execute_izhikevich_step_on_gpu	int	17259
execute_stdp_update_on_gpu	int	17291
execute_stdp_trace_update_on_gpu	int	17320
execute_lbm_collide_and_stream_on_gpu	int	17348
execute_nbody_calculate_forces_on_gpu	int	17377
execute_nbody_integrate_on_gpu	int	17399
execute_ising_metropolis_step_on_gpu	int	17421
execute_shape_loss_with_reward_penalty_list_gpu	int	17449

Anhang B – OpenCL-Kernel-Inventar

Vollständige Liste der erkannten __kernel-Funktionen.

Kernel 1	Kernel 2	Kernel 3
abstract_to_symbolic_concepts_kernel	adam_update	add_bias_mn
add_broadcast_pe	add_elementwise	autonomic_cycle_kernel
brain_bridge_cycle	broadcast_add_bias	cc_gated_agent_decay
cc_mark_active_agents	cc_morphogenetic_rule_step	cc_resonant_field_step
context_resonance_kernel	conv2d_backward_input	conv2d_backward_weight
conv2d_bias_grad	conv2d_forward	cross_entropy_loss_grad
dream_state_generator_kernel	dynamic_token_assignment	embedding_backward_calc_delta_local
embedding_lookup	fused_diffusion	gelu_backward_elementwise
gelu_elementwise	generate_system_narrative_kernel	hebbian_update_local_reduce
intuition_precognition_kernel	ising_metropolis_step	izhikevich_neuron_step
layer_norm	layer_norm_backward	lbm_collide_and_stream
log_softmax_stable_rowwise	matmul_backward_da	matmul_backward_db
matmul_batched	matmul_batched_backward_da	matmul_batched_backward_db
matrix_multiply	mul_backward	mul_elementwise
mycel_colony_update	mycel_diffuse_decay	mycel_nutrient_update
mycel_reinforce	nbody_calculate_forces	nbody_integrate
pairwise_similarity_dot	patch_permute_reshape	patch_permute_reshape_backward
proto_segmented_sum_atomic	proto_update_step	qualia_resonator_kernel
quantum_apply_controlled_not	quantum_apply_controlled_phase	quantum_apply_single_qubit
quantum_compute_probabilities	quantum_expectation_pauli_z	quantum_modular_exponentiation
quantum_phase_flip_except_zero	quantum_phase_oracle	quantum_swap_qubits
reduce_sum_axis01	render_debug	render_frame_buf
render_frame_img	shape_loss_reward_penalty	shape_loss_reward_penalty_list
softmax_backward	softmax_rowwise	sqse_decrypt
sqse_encrypt	stdp_update_step	stdp_update_traces
subqg_inject_agents	subqg_simulation_step	threshold_spike
transformation_planner_kernel	transpose	transpose_12_batched
transpose_backward	transpose_batched_last_two	update_genetic_agents_kernel
vqe_gradient_batch_kernel		